

# Outlines

- Introduction
- Try, catch, throw
- Re-throwing an exception
- this pointer

# Introduction

**Exception:** An exception is an unexpected problem that arises during the execution of a program.

- It disrupts the normal flow of instructions of the program.
- Our program terminates suddenly with some errors/issues. Exception occurs during the running of the program (**runtime**).

**Examples:** Dividing a number by zero, not being able to open a file etc.

The process of handling these types of errors in C++ is known as exception handling.

**Exception handling** is the process of responding to the occurrence of exceptions, allowing the program to continue or terminate gracefully.

# Why Exception Handling is Necessary

**Improves Program Robustness:** Ensures the program can handle unexpected errors without crashing.

**Enhances Code Maintainability:** Separates error-handling code from regular code, making the program easier to read and maintain.

**Facilitates Debugging:** Helps identify and isolate problematic parts of the code.

**User Experience:** Provides meaningful error messages to users, enhancing their experience.

**Resource Management:** Ensures proper release of resources (e.g., memory, file handles) even when an error occurs.

# Way/Mechanism to Handle Exceptions

In C++, we handle exceptions with the help of the **try** and **catch** blocks, along with the **throw** keyword.

**try block**- contains the code that may raise an exception

**throw keyword**- throws an exception when an error is detected

**catch block**- contains the code that handles the exception thrown by the **throw** keyword

# Syntax for Exception Handling

```
try {  
    // code that may raise an exception  
    throw argument;  
}  
  
catch (exception) {  
    // code to handle exception  
}
```

Here, we place the code that might generate an exception inside the **try** block. Every **try** block is followed by the **catch** block.

When an exception occurs, the **throw** statement throws an exception, which is caught by the **catch** block.

The **catch** block cannot be used without the **try** block.

## Example program: Divide by Zero

```
// program to divide two numbers
// throws an exception when the divisor is 0

#include <iostream>
using namespace std;

int main() {

    double numerator, denominator, divide;

    cout << "Enter numerator: ";
    cin >> numerator;

    cout << "Enter denominator: ";
    cin >> denominator;
```

```
try {

    // throw an exception if denominator is 0
    if (denominator == 0)
        throw 0;

    // not executed if denominator is 0
    divide = numerator / denominator;
    cout << "Result = " << divide << endl;

}

catch (int num_exception) {

    cout << "Error: Cannot divide by " <<
num_exception << endl;

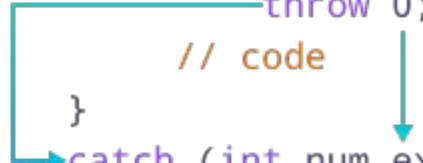
}

return 0;

}
```

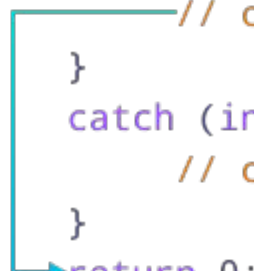
**denominator == 0**

```
try {  
    if (denominator == 0)  
        throw 0;  
    // code  
}  
→ catch (int num_exception) {  
    // code  
}  
return 0;
```



**denominator != 0**

```
try {  
    if (denominator == 0)  
        throw 0;  
    // code  
}  
→ catch (int num_exception) {  
    // code  
}  
return 0;
```



# Re-throw an Exception

**Rethrowing an exception** in C++ involves catching an exception within a **try** block and then throwing it again using the **throw** statement.

This allows us to perform some actions before passing the exception to a higher level in the call stack where it can be handled appropriately.

An exception can only be re-thrown from within a catch block (or from any function call from within that block).

When we re-throw an exception, it will not be re-caught by the same catch block (statement). It will propagate to an outer (next) catch statement (block).

# Why Re-throw an Exception?

- **Perform additional actions:** You can log errors, clean up resources, or modify the exception object before rethrowing.
- **Handle specific exceptions:** You can catch specific exceptions and rethrow others to be handled at a higher level.
- **Change exception type:** While not recommended in most cases, you can create a new exception object and throw it, effectively changing the exception type.

**When to re-throw:** If we need to perform local handling (e.g., logging or cleanup) but want to let higher-level code handle the error more comprehensively.

# Example Program: Re-throw

```
#include <iostream>
#include <stdexcept> // For std::runtime_error

using namespace std;

void process() {
    throw runtime_error("Something went wrong in process");
}

void function() {
    try {
        process(); // Call process, which throws an
exception
    } catch (exception& e) {
        cout << "Caught in function: " << e.what() << endl;
        // Perform some additional handling or logging
        throw; // Re-throw the exception to be handled by
the caller
    }
}
```

```
int main() {
    try {
        function(); // Call function,
which may re-throw an exception
    } catch (exception& e) {
        cerr << "Caught in main: " <<
e.what() << endl;
    }

    return 0;
}
```

## Q. Dec, 2018

Write a program to read three numbers X, Y, and Z and evaluate R where  $R = Z / (X - Y)$ . Use exception handling to throw an exception in case division by zero is attempted.

```
#include <iostream>
using namespace std;

int main() {
    double X, Y, Z, R;

    // Read input from the user
    cout << "Enter three numbers X, Y, and Z: ";
    cin >> X >> Y >> Z;
    try {
        double denominator = X - Y;
        if (denominator == 0) {
            throw 0;
        }
        R = Z / denominator;
        cout<<"Result: "<< R << endl;
    }
    catch (int num_exception) {
        cout << "Error: Cannot divide by " << num_exception << endl;
    }
    return 0;
}
```

## Additional Topic: this pointer

### What is the `this` pointer?

In C++, the `this` pointer is a special keyword that refers to the current object instance of a class. It's automatically passed as an implicit argument to all non-static member functions. Essentially, it points to the object itself.

### How does it work?

When you call a member function on an object, the compiler automatically passes the address of that object to the function as the `this` pointer. This allows the function to access and modify the object's data members and member functions.

## Key Points

- The **this** pointer is a constant pointer. You cannot change its value.
- It's only available within non-static member functions. Static member functions don't have a **this** pointer because they can be called without an object instance.
- You can use the **this** pointer to differentiate between local variables and member variables with the same name.

## Use cases

- It can be used to refer current class instance variable.
- Accessing member variables within member functions
- Passing the current object as an argument to other functions

## Example program

```
#include <iostream>
using namespace std;
class Employee {
public:
    int id;
    string name;
    float salary;
```

```
Employee(int i, string n, float s){
    this->id = i;
    this->name = n;
    this->salary = s;
}
```

```
void display(){
    cout<<this->id<<"-"<<this->name<<"-"<<
    this->salary<<endl;
```

```
};
```

```
int main(void) {
    Employee e1 = Employee(101, "Sonoo", 890000);
    Employee e2 = Employee(102, "Nakul", 59000);
    e1.display();
    e2.display();
    return 0;
}
```



Any Queries?